

NEURAL NETWORK MECHANICS

- A **neuron**: output = activation($w \cdot x + b$). Layers of neurons; the **non-linear activation** is what lets the network learn curves (stacked linear layers collapse to one linear map).
- Training loop**: forward pass → loss → **backpropagation** (chain rule gives $d\text{Loss}/dw$ for every weight) → optimiser step (Adam/SGD) → repeat per mini-batch; one pass over the data = epoch.

Activation	Formula / range	Note
ReLU	$\max(0, x)$	default hidden; "dying ReLU"
Leaky ReLU / GELU	small slope for $x < 0$ / smooth	transformers use GELU
Sigmoid	$1/(1 + e^{-(x)})$, (0, 1)	binary output; vanishing gradients
Tanh	(-1, 1)	zero-centred; RNNs
Softmax	$e^{(z_i)} / \sum e^{(z_j)}$	multiclass probabilities

- Output + loss pairs**: regression: linear + MSE; binary: sigmoid + binary cross-entropy; multiclass: softmax + cross-entropy.
- Optimisers**: SGD + momentum; **Adam** (default); learning-rate schedules, warm-up; typical Adam lr $1e-3$ ($1e-4$ for fine-tuning).
- Initialisation**: Xavier/Glorot (sigmoid/tanh), He (ReLU). **Vanishing/exploding gradients**: ReLU, good init, **batch/layer norm**, residual connections, gradient clipping.
- Regularise**: dropout (randomly zero units while training), weight decay (L2), early stopping, data augmentation, more data, smaller net.
- Batch norm** normalises activations per batch (faster, stabler); **layer norm** per example (transformers).
- For small tabular data, gradient boosting usually beats deep nets.

ARCHITECTURES AT A GLANCE

Type	Good for	Core idea
MLP	tabular, simple	dense layers
CNN	images, audio	local filters with shared weights, pooling
RNN / LSTM / GRU	sequences	hidden state; LSTM gates fight vanishing gradients
Transformer	text, now vision	self-attention, parallel
Autoencoder / VAE	compression, anomalies, generation	encoder → latent → decoder
GAN / diffusion	image generation	adversarial / denoising

- CNN output size**: $(W - K + 2P)/S + 1$. Parameters of a conv layer: $K \cdot K \cdot C_{in} \cdot C_{out} + C_{out}$.
- Framework skeleton (PyTorch): `loss = crit(model(x), y); opt.zero_grad(); loss.backward(); opt.step(); model.eval()` and `torch.no_grad()` for inference.

BACKPROP AND SEQUENCE MODELS BY HAND

- One neuron, squared loss: $y_{\text{hat}} = \sigma(wx + b)$, $L = (y_{\text{hat}} - y)^2$; $dL/dw = 2(y_{\text{hat}} - y) \cdot \sigma'(z) \cdot x$; update $w \leftarrow w - \eta \cdot dL/dw$. Backprop reuses each layer's gradient going backwards.
- RNN**: $h_t = \tanh(W \cdot h_{t-1} + U \cdot x_t)$; gradients shrink over long sequences. **LSTM** adds forget, input and output gates and a cell state; **GRU** is a lighter two-gate version.
- Parameter count of a dense layer: $n_{in} \cdot n_{out} + n_{out}$.
- Hyperparameters**: learning rate (most important), batch size, depth/width, dropout rate, weight decay, epochs with early stopping.
- Debug order: data and labels → loss at start ($\sim \ln(K)$ for K classes) → overfit one batch → learning rate → architecture.

COMPUTER VISION

- Convolution** slides small filters (edges → textures → parts → objects); pooling shrinks maps. Far fewer parameters than dense layers.
- Tasks: **classification** (label), **detection** (boxes + labels: YOLO, Faster R-CNN), **segmentation** (label per pixel: U-Net, Mask R-CNN).
- Transfer learning**: take a model pretrained on ImageNet, replace the head, freeze early layers, fine-tune the rest. Works with small datasets. **Augmentation**: flips, crops, colour jitter.
- IoU** = overlap area/union area; 2 boxes of area 16 overlapping 8: $8/24 = 0.33$. A detection counts correct at $\text{IoU} \geq 0.5$. **AP** = area under the precision-recall curve per class; **mAP** = mean over classes (and IoU thresholds).
- Check train and production images look alike (lighting, devices).

CLASSICAL NLP PIPELINE

- Clean → tokenise → (lowercase, stop words, stem/lemmatise) → vectorise → model.
- Bag of words**: counts. **TF-IDF** = $\text{tf} \times \log(N/\text{df})$: frequent in this document, rare overall; sparse, ignores order and meaning. n-grams capture short phrases.
- Embeddings**: dense vectors where similar meanings are close (word2vec, GloVe, fastText); **contextual** embeddings (BERT) give "bank" different vectors by sentence. Cosine similarity compares vectors.
- Baselines: TF-IDF + logistic regression / linear SVM / Naive Bayes are strong and cheap; then fine-tune a transformer.
- Tasks and metrics: classification (F1), NER, summarisation (ROUGE), translation (BLEU), retrieval (recall@K, MRR), language modelling (perplexity).

```
from sklearn.feature_extraction.text
import (
    TfidfVectorizer)
v = TfidfVectorizer(ngram_range=(1, 2),
    min_df=2, stop_words="english")
X = v.fit_transform(texts)
```

TRANSFORMERS AND ATTENTION

- **Self-attention:** each token makes a query Q, key K and value V. Attention = $\text{softmax}(Q \cdot K^T / \sqrt{d_k}) \cdot V$: every token takes a weighted average of all tokens' values, weights from query-key match. The $\sqrt{d_k}$ scaling keeps softmax from saturating.
- **Multi-head:** several attentions in parallel learn different relations. **Positional encodings** inject order. **Residual + layer norm**, feed-forward block per layer.
- **Encoder** (BERT: understanding, classification, embeddings), **decoder** (GPT: next-token generation, causal mask), encoder-decoder (T5: translation, summarisation).
- vs RNNs: parallel training, direct long-range links. **Cost grows as n^2 with sequence length**, which limits context.
- Pretrain on huge text → **fine-tune** for a task.

LLM FUNDAMENTALS

- **Tokens:** sub-word pieces (~0.75 words each); context window = max tokens in prompt + output. Cost and latency scale with tokens.
- Generation: the model outputs a probability over the next token. **Temperature 0** = near-deterministic, higher = more varied; **top-p** (nucleus) samples from the most likely mass; **top-k**.
- Training stages: pretraining (next token) → **instruction tuning (SFT)** → **RLHF / preference tuning (DPO)**.
- **Prompting:** clear role, task and format; **few-shot** examples; ask for step-by-step reasoning (chain of thought); delimit inputs; request JSON and validate it; keep prompts versioned and tested.
- **Hallucination:** fluent but false output. Reduce with retrieval (RAG) and citations, lower temperature, constrained formats, verification/tool calls, "say you do not know".
- **Evaluation:** a golden set of real questions; automatic checks, LLM-as-judge (with human calibration); track regressions per prompt/model change.
- Safety: prompt injection (untrusted text containing instructions), PII leakage, jailbreaks: filter, isolate untrusted content, least-privilege tools.

RAG END TO END

- **Offline:** load → clean → **chunk** (200-800 tokens, overlap 10-20%, respect headings) → **embed** → store in a vector index (FAISS, pgvector, Pinecone) with metadata.
- **Online:** embed query → retrieve top-k (similarity: cosine/dot; ANN indexes: HNSW, IVF) → optional **rerank** (cross-encoder) → build prompt with sources and "answer only from the context" → generate with citations.
- Improve: **hybrid search** (BM25 keyword + vectors), metadata filters, query rewriting, parent-document retrieval, good chunking.
- **Evaluate separately:** retrieval (recall@K, hit rate, MRR) and generation (faithfulness/groundedness, answer relevance, correctness). If the answer is wrong, **check retrieval first**.
- Failure modes: wrong chunk size, stale index, missing documents, embedding model mismatch, context overflow, lost-in-the-middle.

RAG VS FINE-TUNING, AGENTS

Need	Use
Fresh, private or changing facts, citations	RAG
Style, tone, format, specialised behaviour, lower latency	Fine-tune (LoRA / QLoRA: train small low-rank adapters)
Both	RAG + a lightly fine-tuned model

- Order of effort: prompting → RAG → fine-tuning.
- **Agent:** an LLM that plans and calls tools (search, SQL, code, APIs) in a loop. ReAct pattern: reason → act → observe. Add step limits, logging, validation, human approval for risky actions; errors compound over steps.
- **Embedding models** for search vs **generation models:** different jobs. **Function calling / structured outputs** make tool use reliable.
- Project ideas: document Q&A bot, support-ticket triage, text-to-SQL assistant, resume screener, meeting summariser (with evaluation set and cost report).
- Cost levers: smaller model, caching, shorter prompts, batching, streaming.

TOKENISATION, EMBEDDINGS, PROMPTS

- **BPE / WordPiece** tokenisers split rare words into pieces: no out-of-vocabulary words; counts differ by language (non-English costs more tokens).
- Sentence embeddings (e.g. SBERT) → cosine similarity for semantic search, clustering, dedupe; normalise vectors so dot product = cosine.
- Prompt skeleton: **Role** (you are ...), **Task** (do ...), **Context** (use only ...), **Format** (JSON with keys ...), **Examples** (2-3), **Constraints** (if unknown say so).
- Eval table: question | expected | retrieved | answer | grounded? | correct? → aggregate; re-run after every prompt, chunking or model change.
- Rough sizes: context 8k-200k tokens; model parameter count ~ memory (7B params ~ 14 GB in fp16, ~4 GB with 4-bit quantisation).

TRAINING TIPS AND PITFALLS

- Overfit a tiny batch first to check the pipeline; watch train vs validation loss; stop when validation stalls.
- Loss NaN: lower the learning rate, check inputs, clip gradients. Class imbalance: weights or sampling.
- Data leakage via near-duplicates between train and test (images, text).
- GPU memory: smaller batch, mixed precision (fp16/bf16), gradient accumulation.
- Reproducibility: seeds, pinned versions, logged configs.

SAY IT IN 20 SECONDS

- "Attention lets each token take a weighted average of all tokens."
- "RAG adds knowledge at question time; fine-tuning changes behaviour."
- "If a RAG answer is wrong, check retrieval before the prompt."
- "ReLU gives non-linearity; backprop is the chain rule."
- "Transfer learning: pretrained backbone, new head, fine-tune."